FRD

Functional Requirements Document (FRD)

AWS S3-Based Data Processing Pipeline

•

FR-INGEST-001

• Title: Ingest Customer CSV from S3

• Description:

The system shall read customer CSV files from the designated S3 input path and load them into the processing pipeline.

• Preconditions:

• Customer CSV files exist at `s3://adif-sdlc/sdlc_wizard/customerdata/`

• Files conform to schema: CustId, Name, EmailId, Region

• AWS Glue job has read permissions on the S3 bucket

• Main Flow / Functional Steps:

• Glue job reads all CSV files from `s3://adif-sdlc/sdlc_wizard/customerdata/`

• Parse CSV with schema: CustId, Name, EmailId, Region

• Load data into Spark DataFrame for downstream processing

• Postconditions:

• Customer data is available in memory for cleaning and transformation

• No data is written to target tables at this stage

• Assumptions:

• CSV files use standard delimiters (comma)

• Header row is present in each file

• All files in the path are valid customer data files

•

FR-INGEST-002

• Title: Ingest Order CSV from S3

• Description:

The system shall read order CSV files from the designated S3 input path and load them into the processing pipeline.

- Preconditions:
- Order CSV files exist at `s3://adif-sdlc/sdlc_wizard/orderdata/`
- Files conform to schema: OrderId, ItemName, PricePerUnit, Qty, Date, CustId
- AWS Glue job has read permissions on the S3 bucket

- Main Flow / Functional Steps:
- Glue job reads all CSV files from `s3://adif-sdlc/sdlc_wizard/orderdata/`
- Parse CSV with schema: OrderId, ItemName, PricePerUnit, Qty, Date, CustId
- Load data into Spark DataFrame for downstream processing

- Postconditions:
- Order data is available in memory for cleaning and transformation
- No data is written to target tables at this stage

- Assumptions:
- CSV files use standard delimiters (comma)
- Header row is present in each file
- All files in the path are valid order data files

-

FR-CLEAN-001

- Title: Remove Null Values from Customer Data

- Description:

The system shall remove all rows containing "Null" (string) or NULL values from the customer dataset.

- Preconditions:
- Customer data is loaded into Spark DataFrame
- Schema includes: CustId, Name, EmailId, Region

- Main Flow / Functional Steps:
- Scan all columns in customer DataFrame
- Identify rows where any column contains "Null" (case-sensitive string) or NULL
- Drop identified rows from the DataFrame

- Postconditions:
- Customer DataFrame contains no "Null" or NULL values

- Cleaned data is ready for duplicate removal

- Assumptions:
- "Null" refers to the literal string "Null"
- NULL refers to Spark null values
- Partial row removal is acceptable (entire row dropped if any column is null)

-

FR-CLEAN-002

- Title: Remove Null Values from Order Data

- Description:

The system shall remove all rows containing "Null" (string) or NULL values from the order dataset.

- Preconditions:
- Order data is loaded into Spark DataFrame
- Schema includes: OrderId, ItemName, PricePerUnit, Qty, Date, CustId

- Main Flow / Functional Steps:
- Scan all columns in order DataFrame
- Identify rows where any column contains "Null" (case-sensitive string) or NULL
- Drop identified rows from the DataFrame

- Postconditions:
- Order DataFrame contains no "Null" or NULL values
- Cleaned data is ready for duplicate removal

- Assumptions:
- "Null" refers to the literal string "Null"
- NULL refers to Spark null values
- Partial row removal is acceptable (entire row dropped if any column is null)

-

FR-CLEAN-003

- Title: Remove Duplicate Records from Customer Data

- Description:

The system shall identify and remove duplicate rows from the customer dataset.

- Preconditions:
- Customer data has been cleaned of null values
- DataFrame contains schema: CustId, Name, EmailId, Region

- Main Flow / Functional Steps:
- Apply deduplication logic across all columns in customer DataFrame
- Retain first occurrence of duplicate rows
- Drop subsequent duplicate rows

- Postconditions:
- Customer DataFrame contains only unique records
- Data is ready for catalog registration

- Assumptions:
- Duplicates are identified by comparing all column values
- First occurrence is retained by default Spark behavior

-

FR-CLEAN-004

- Title: Remove Duplicate Records from Order Data

- Description:

The system shall identify and remove duplicate rows from the order dataset.

- Preconditions:
- Order data has been cleaned of null values
- DataFrame contains schema: OrderId, ItemName, PricePerUnit, Qty, Date, CustId

- Main Flow / Functional Steps:
- Apply deduplication logic across all columns in order DataFrame
- Retain first occurrence of duplicate rows
- Drop subsequent duplicate rows

- Postconditions:
- Order DataFrame contains only unique records
- Data is ready for catalog registration

- Assumptions:
- Duplicates are identified by comparing all column values

- First occurrence is retained by default Spark behavior
-

FR-CATALOG-001

- Title: Register Customer Table in Glue Data Catalog
- Description:

The system shall register the cleaned customer data as a table in the AWS Glue Data Catalog.

- Preconditions:
- Customer data is cleaned (nulls and duplicates removed)
- Glue database `gen_ai_poc_databrickscoe` exists
- Glue job has permissions to create/update catalog tables
- Main Flow / Functional Steps:
- Write cleaned customer DataFrame to Glue Catalog
- Database: `gen_ai_poc_databrickscoe`
- Table name: `sdlc_wizard_customer`
- Schema: CustId, Name, EmailId, Region
- Postconditions:
- Table `sdlc_wizard_customer` is queryable via Athena
- Metadata is registered in Glue Data Catalog
- Assumptions:
- Table is created if it does not exist
- Existing table is overwritten or appended based on job configuration
-

FR-CATALOG-002

- Title: Register Order Table in Glue Data Catalog
- Description:

The system shall register the cleaned order data as a table in the AWS Glue Data Catalog.

- Preconditions:
- Order data is cleaned (nulls and duplicates removed)
- Glue database `gen_ai_poc_databrickscoe` exists
- Glue job has permissions to create/update catalog tables

- Main Flow / Functional Steps:
- Write cleaned order DataFrame to Glue Catalog
- Database: `gen_ai_poc_databrickscoe`
- Table name: `sdlc_wizard_order`
- Schema: OrderId, ItemName, PricePerUnit, Qty, Date, CustId

- Postconditions:
- Table `sdlc_wizard_order` is queryable via Athena
- Metadata is registered in Glue Data Catalog

- Assumptions:
- Table is created if it does not exist
- Existing table is overwritten or appended based on job configuration

-

FR-SCD2-001

- Title: Implement SCD Type 2 for Order Summary Table

- Description:

The system shall maintain a Slowly Changing Dimension Type 2 table (`ordersummary`) using Apache Hudi on S3, tracking historical changes to customer and order data.

- Preconditions:
- Customer and order tables are registered in Glue Catalog
- Target S3 path `s3://adif-sdlc/curated/sdlc_wizard/ordersummary/` is accessible
- Hudi libraries are available in Glue job environment

- Main Flow / Functional Steps:
- Join customer and order DataFrames on CustId
- Compare joined result with existing `ordersummary` table
- For changed records:
- Set IsActive = False, EndDate = current timestamp for existing row
- Insert new row with IsActive = True, StartDate = current timestamp
- For new records:
- Insert with IsActive = True, StartDate = current timestamp, EndDate = NULL
- Write results to `s3://adif-sdlc/curated/sdlc_wizard/ordersummary/` using Hudi format

- Postconditions:

- `ordersummary` table reflects current and historical states
- All active records have IsActive = True
- Closed records have IsActive = False with populated EndDate
- OpTs (operation timestamp) is recorded for all operations

- Assumptions:
- Primary key for SCD2 comparison is derived from joined customer-order data
- OpTs is system-generated timestamp
- EndDate is NULL for active records

-

FR-SCD2-002

- Title: Register Order Summary Table in Glue Data Catalog

- Description:

The system shall register the `ordersummary` Hudi table in the AWS Glue Data Catalog.

- Preconditions:
- `ordersummary` data is written to S3 in Hudi format
- Glue database `gen_ai_poc_databrickscoe` exists

- Main Flow / Functional Steps:
- Register Hudi table at `s3://adif-sdlc/curated/sdlc_wizard/ordersummary/`
- Database: `gen_ai_poc_databrickscoe`
- Table name: `ordersummary`
- Include SCD2 columns: IsActive, StartDate, EndDate, OpTs

- Postconditions:
- `ordersummary` table is queryable via Athena
- Hudi metadata is accessible through Glue Catalog

- Assumptions:
- Hudi table format is compatible with Glue Catalog
- Athena supports querying Hudi tables

-

FR-INCR-001

- Title: Trigger Incremental SCD2 Updates on Customer Changes

- Description:

The system shall detect changes in customer data and trigger incremental updates to the `ordersummary` table using SCD Type 2 logic.

- Preconditions:
- New or updated customer data is available in `sdlc_wizard_customer` table
- Existing `ordersummary` table is accessible
- Hudi incremental query capabilities are enabled

- Main Flow / Functional Steps:
- Identify changed or new customer records since last processing
- Join changed customer records with order data
- Apply SCD2 logic to `ordersummary`:
- Close previous active rows (IsActive = False, set EndDate)
- Insert new active rows (IsActive = True, set StartDate)
- Update OpTs for all affected rows

- Postconditions:
- `ordersummary` reflects customer data changes
- Historical records are preserved with proper SCD2 flags
- IsActive, StartDate, EndDate, OpTs are maintained correctly

- Assumptions:
- Change detection is based on customer data comparison (e.g., CustId + hash of other fields)
- Incremental processing runs on a scheduled or event-driven basis

-

FR-AGG-001

- Title: Generate Customer Aggregate Spend Table

- Description:

The system shall create an aggregated table (`customeraggregatespend`) summarizing total spending per customer.

- Preconditions:
- `ordersummary` table is available and up-to-date
- Target S3 path `s3://adif-sdlc/analytics/customeraggregatespend/` is accessible

- Main Flow / Functional Steps:

- Read active records from `ordersummary` (IsActive = True)
- Calculate TotalAmount per customer: SUM(PricePerUnit Qty) grouped by Name
- Include Date field (latest order date or aggregation date)
- Write result to `s3://adif-sdlc/analytics/customeraggregatespend/`
- Schema: Name, TotalAmount, Date

- Postconditions:
- `customeraggregatespend` table contains aggregated spending data
- Data is stored at `s3://adif-sdlc/analytics/customeraggregatespend/`

- Assumptions:
- TotalAmount is calculated as PricePerUnit Qty summed per customer
- Date represents the date of aggregation or latest order date
- Only active records from `ordersummary` are included

-

FR-AGG-002

- Title: Register Customer Aggregate Spend Table in Glue Data Catalog

- Description:

The system shall register the `customeraggregatespend` table in the AWS Glue Data Catalog.

- Preconditions:
- Aggregated data is written to `s3://adif-sdlc/analytics/customeraggregatespend/`
- Glue database `gen_ai_poc_databrickscoe` exists

- Main Flow / Functional Steps:
- Register table at `s3://adif-sdlc/analytics/customeraggregatespend/`
- Database: `gen_ai_poc_databrickscoe`
- Table name: `customeraggregatespend`
- Schema: Name, TotalAmount, Date

- Postconditions:
- `customeraggregatespend` table is queryable via Athena
- Metadata is registered in Glue Data Catalog

- Assumptions:
- Table format is compatible with Athena queries
- Table is created or updated based on job configuration

•

FR-ARCH-001

• Title: Use AWS Glue for ETL Processing

• Description:

The system shall use AWS Glue with Spark and Hudi for all ETL operations including ingestion, cleaning, transformation, and SCD2 logic.

• Preconditions:

• AWS Glue service is enabled in the account

• Glue jobs have necessary IAM permissions for S3, Glue Catalog, and CloudWatch

• Main Flow / Functional Steps:

• Execute all data processing steps within AWS Glue Spark jobs

• Use Hudi for SCD Type 2 table management

• Log job execution details to CloudWatch

• Postconditions:

• All ETL operations are executed via Glue

• Job metrics and logs are available in CloudWatch

• Assumptions:

• Glue version supports Hudi libraries

• Spark configuration is optimized for workload size

•

FR-ARCH-002

• Title: Query Data Using Amazon Athena

• Description:

The system shall enable querying of all registered Glue Catalog tables using Amazon Athena.

• Preconditions:

• Tables are registered in Glue Data Catalog

• Athena has access to underlying S3 data

• Query results S3 bucket is configured

• Main Flow / Functional Steps:

- Users execute SQL queries via Athena console or API
- Athena reads metadata from Glue Data Catalog
- Athena scans data from S3 paths
- Query results are returned to user or written to S3

- Postconditions:
- All catalog tables are queryable via standard SQL
- Query results are accessible

- Assumptions:
- Athena supports file formats used (CSV, Hudi, Parquet, etc.)
- Users have appropriate IAM permissions for Athena and S3

-

FR-ARCH-003

- Title: Monitor ETL Jobs with CloudWatch

- Description:

The system shall log and monitor all Glue job executions using AWS CloudWatch.

- Preconditions:
- CloudWatch logging is enabled for Glue jobs
- Glue jobs have permissions to write logs to CloudWatch

- Main Flow / Functional Steps:
- Glue jobs write execution logs to CloudWatch Logs
- Capture job start time, end time, status, and error messages
- Enable CloudWatch metrics for job duration and data processed

- Postconditions:
- All job executions are logged in CloudWatch
- Metrics are available for monitoring and alerting

- Assumptions:
- CloudWatch retention policies are configured
- Alerts can be configured based on job failures or performance thresholds

-

FR-ARCH-004

- Title: Optional Data Governance with AWS Lake Formation

- Description:

The system may optionally use AWS Lake Formation for data governance, access control, and auditing.

- Preconditions:
- Lake Formation is enabled in the AWS account
- Glue Catalog is integrated with Lake Formation

- Main Flow / Functional Steps:
- Register S3 data locations with Lake Formation
- Define fine-grained access policies for tables and columns
- Enforce permissions through Lake Formation instead of S3 bucket policies

- Postconditions:
- Data access is governed by Lake Formation policies
- Audit logs are available in Lake Formation

- Assumptions:
- Lake Formation is optional and not required for core functionality
- If enabled, all data access goes through Lake Formation permissions

-

ASSUMPTIONS

- All S3 paths are accessible and correctly configured with IAM permissions
- CSV files contain headers matching the specified schemas
- Hudi is configured correctly in the Glue environment
- SCD2 logic assumes a composite key or unique identifier for change detection
- Date fields in order data are in a parseable format
- TotalAmount calculation assumes PricePerUnit and Qty are numeric
- Incremental processing frequency is defined outside this FRD
- CloudWatch log retention and alerting rules are configured separately
- Lake Formation is optional and not required for MVP
- No data encryption or masking requirements are specified in the BRD coding section
- All AWS services are deployed in the same region